

SQL i nie tylko

Wykład piąty

dr inż. Marcin Ciura, Uniwersytet Komisji Edukacji Narodowej

Proszę zrobić
notatki :-)

Plan na dziś

Plan na dziś

Z tego wykładu dowiedziecie się:

- + jak sortować wyniki zapytań dzięki klauzuli **ORDER BY**
- + jak ograniczać liczbę wierszy w wynikach dzięki klauzuli **LIMIT**
- + jak pisać złączenia zewnętrzne, czyli **OUTER JOIN**
- + jak pisać zapytania zagnieżdżone

Przepraszam was za to, że dziś znów
nie będzie dowcipu na początek :-)

Klauzula ORDER BY

Klauzula ORDER BY

Klauzula ORDER BY sortuje wyniki zapytania

Bez tej klauzuli system zarządzania bazą danych zwraca wiersze w trudnej do przewidzenia kolejności

Klauzula ORDER BY

Umieszczamy klauzulę ORDER BY na końcu zapytania, po klauzulach GROUP BY i HAVING, a przed klauzulą LIMIT

SELECT	Student
FROM	filozofii
WHERE	wesoło
GROUP BY	głosi
HAVING	hasła
ORDER BY	o
LIMIT	ludziach



Klauzula ORDER BY

Możemy sortować dane w dwóch kierunkach:

- + ASC – rosnąco (ascending), czyli od A do Z, od najmniejszej do największej liczby. ASC to domyślny kierunek sortowania, czyli nie trzeba go dopisywać do zapytania
- + DESC – malejąco (descending), czyli od Z do A, od największej do najmniejszej liczby

Klauzula ORDER BY

Przykład 1: Wypisz wypłaty od najwyższej do najniższej

```
SELECT id_pracownika, kwota_wypłaty_PLN
FROM PracownikZespołu
ORDER BY kwota_wypłaty_PLN DESC;
```

id_pracownika	kwota_wypłaty_PLN
4	20000.0
1	15000.0
2	15000.0
1	12000.0
2	10000.0
1	NULL
2	NULL
3	NULL

Klauzula ORDER BY

Możemy sortować dane według kilku kolumn jednocześnie

System zarządzania bazą danych najpierw posortuje wiersze według pierwszej kolumny. W przypadku remisów użyje drugiej kolumny, potem trzeciej kolumny i tak dalej

Możemy określić kierunek sortowania dla każdej kolumny

Klauzula ORDER BY

Przykład 2: Wypisz pracowników i ich wypłaty od najwyższej do najniższej. Jeśli dwie wypłaty są równe, wypisz pracowników alfabetycznie

```
SELECT imię, nazwisko, kwota_wypłaty_PLN
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)
ORDER BY kwota_wypłaty_PLN DESC, nazwisko ASC, imię ASC;
```

imię	nazwisko	kwota_wypłaty_PLN
Lucyna	Lis	20000.0
Olga	Owca	15000.0
Wiktor	Wilk	15000.0
Olga	Owca	12000.0
Wiktor	Wilk	10000.0
Kasia	Kot	NULL
Olga	Owca	NULL
Wiktor	Wilk	NULL

Klauzula ORDER BY

Klauzula ORDER BY jest wykonywana po klauzuli SELECT, dlatego możemy używać w niej aliasów

FROM	Filip
WHERE	wolno
GROUP BY	gryzie
HAVING	hamburgera
SELECT	stojąc
DISTINCT	dumnie
ORDER BY	obok
LIMIT	ładny



Klauzula ORDER BY

Przykład 3: Wypisz w kolejności malejącej podatek od wypłat

```
SELECT id_pracownika, kwota_wypłaty_PLN * 0.23 AS podatek
FROM PracownikZespołu
ORDER BY podatek DESC;
```

id_pracownika	podatek
4	4600.0
1	3450.0
2	3450.0
1	2760.0
2	2300.0
1	NULL
2	NULL
3	NULL

Klauzula ORDER BY

Zamiast wypisywać w klauzuli ORDER BY nazwy kolumn, możemy podawać numery ich pozycji w klauzuli SELECT. Te pozycje są numerowane od 1

Klauzula ORDER BY

Przykład 4: Wypisz pracowników i ich wypłaty od najwyższej do najniższej. Jeśli dwie wypłaty są równe, wypisz pracowników alfabetycznie. Użyj numerów kolumn w klauzuli ORDER BY

```
SELECT imię, nazwisko, kwota_wypłaty_PLN
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)
ORDER BY 3 DESC, 2 ASC, 1 ASC; -- kwota_wypłaty_PLN DESC, nazwisko ASC, imię ASC
```

imię	nazwisko	kwota_wypłaty_PLN
Lucyna	Lis	20000.0
Olga	Owca	15000.0
Wiktor	Wilc	15000.0
Olga	Owca	12000.0
Wiktor	Wilc	10000.0
Kasia	Kot	NULL
Olga	Owca	NULL
Wiktor	Wilc	NULL

Klauzula ORDER BY

W SQLite wartości NULL są traktowane jako najmniejsze. W innych systemach zarządzania bazami danych wartości NULL mogą być traktowane inaczej

Aby mieć kontrolę nad tym, gdzie będą wyświetlane wartości NULL, po ASC lub DESC piszemy NULLS FIRST albo NULLS LAST

Klauzula ORDER BY

Przykład 5: Wypisz pracowników i ich wypłaty od najwyższej do najniższej. Jeśli dwie wypłaty są równe, wypisz pracowników alfabetycznie. Jeśli pracownik jeszcze nie dostał wypłaty, wypisz go na początku

```
SELECT imię, nazwisko, kwota_wypłaty_PLN
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)
ORDER BY kwota_wypłaty_PLN DESC NULLS FIRST, nazwisko ASC, imię ASC;
```

imię	nazwisko	kwota_wypłaty_PLN
Kasia	Kot	NULL
Olga	Owca	NULL
Wiktor	Wilk	NULL
Lucyna	Lis	20000.0
Olga	Owca	15000.0
Wiktor	Wilk	15000.0
Olga	Owca	12000.0
Wiktor	Wilk	10000.0

Klauzuļa LIMIT

Klauzula LIMIT

Klauzula LIMIT ogranicza liczbę wierszy
zwracaną przez zapytanie

Klauzula LIMIT

Umieszczamy klauzulę LIMIT na samym końcu zapytania, po klauzuli ORDER BY

SELECT	Student
FROM	filozofii
WHERE	wesoło
GROUP BY	głosi
HAVING	hasła
ORDER BY	o
LIMIT	ludziach



Klauzula LIMIT

Przykład 1: Znajdź najwyższą wypłatę

```
SELECT imię, nazwisko, kwota_wypłaty_PLN  
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)  
ORDER BY kwota_wypłaty_PLN DESC  
LIMIT 1;
```

imię	nazwisko	kwota_wypłaty_PLN
Lucyna	Lis	20000.0

Klauzula LIMIT

Klauzula LIMIT rzadko ma sens bez klauzuli
ORDER BY

Bez klauzuli ORDER BY system zarządzania
bazą danych zwróci takie wiersze, jakie
zechce :-)

Klauzula LIMIT

Za pomocą klauzuli OFFSET w parze z klauzulą LIMIT możemy stronicować wyniki

LIMIT określa, ile wierszy pobrać
OFFSET (przesunięcie) określa, ile początkowych wierszy pominąć

Klauzula LIMIT

Przykład 2: Znajdź 3 najwyższe wypłaty

```
SELECT imię, nazwisko, kwota_wypłaty_PLN  
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)  
ORDER BY kwota_wypłaty_PLN DESC  
LIMIT 3;
```

imię	nazwisko	kwota_wypłaty_PLN
Lucyna	Lis	20000.0
Olga	Owca	15000.0
Wiktor	Wilk	15000.0

Klauzula LIMIT

Przykład 3: Znajdź 3 kolejne wypłaty w kolejności malejącej

```
SELECT imię, nazwisko, kwota_wypłaty_PLN  
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)  
ORDER BY kwota_wypłaty_PLN DESC  
LIMIT 3 OFFSET 3;
```

imię	nazwisko	kwota_wypłaty_PLN
Olga	Owca	12000.0
Wiktor	Wilk	10000.0
Olga	Owca	NULL

Klauzula LIMIT

Przykład 4: Znajdź pozostałe wypłaty w kolejności malejącej (co najwyżej 3)

```
SELECT imię, nazwisko, kwota_wypłaty_PLN  
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)  
ORDER BY kwota_wypłaty_PLN DESC  
LIMIT 3 OFFSET 6;
```

imię	nazwisko	kwota_wypłaty_PLN
Wiktor	Wilc	NULL
Kasia	Kot	NULL

Klauzula LIMIT

Gdy tworzymy skomplikowane zapytanie do dużej bazy danych, dobrą praktyką jest dopisać na końcu tego zapytania **LIMIT 5**

Inaczej system zarządzania bazą danych zwróci nie wiadomo, ile wierszy, przez co sprawdzenie, czy stworzyliśmy poprawne zapytanie, może być niewygodne

Proszę o pytania :-)

Złączenia zewnętrzne,
czyli OUTER JOIN

Złączenia zewnętrzne, czyli OUTER JOIN

Złączenie zewnętrzne (OUTER JOIN) łączy dwie tabele tak, że nie tracimy tych wierszy, które nie mają odpowiednika w drugiej tabeli

System zarządzania bazą danych wstawia w takich miejscach wartości NULL

Złączenia zewnętrzne, czyli OUTER JOIN

Przykład: Dodajmy do naszej bazy danych tabelę Zamówienie

```
SELECT * FROM Zamówienie;
```

id_zamówienia	id_pracownika
101	1
102	1
103	2
104	99

Złączenia zewnętrzne, czyli OUTER JOIN

Przykład 1: Złączenie lewostronne (LEFT JOIN lub LEFT OUTER JOIN) zwraca wszystkie wiersze z lewej tabeli i dopasowane wiersze z prawej tabeli. Jeśli w lewej tabeli są takie wiersze, które nie pasują do niczego w prawej tabeli, to kolumny prawej tabeli są uzupełniane wartościami NULL

```
SELECT imię, id_zamówienia  
FROM Pracownik LEFT JOIN Zamówienie USING (id_pracownika);
```

imię	id_zamówienia
Olga	101
Olga	102
Wiktor	103
Kasia	NULL
Lucyna	NULL

Złączenia zewnętrzne, czyli OUTER JOIN

Przykład 2: Złączenie prawostronne (RIGHT JOIN lub RIGHT OUTER JOIN) zwraca wszystkie wiersze z prawej tabeli i dopasowane wiersze z lewej tabeli. Brakujące dane z lewej tabeli są uzupełniane wartościami NULL

```
SELECT imię, id_zamówienia  
FROM Pracownik RIGHT JOIN Zamówienie USING (id_pracownika);
```

imię	id_zamówienia
Olga	101
Olga	102
Wiktor	103
NULL	104

Złączenia zewnętrzne, czyli OUTER JOIN

Przykład 3: Złączenie obustronne (FULL JOIN lub FULL OUTER JOIN) zwraca wszystkie wiersze z obu tabel. Jeśli istnieje dopasowanie – łączy dane w jeden wiersz. Jeśli nie – uzupełnia brakującą stronę wartościami NULL

```
SELECT imię, id_zamówienia  
FROM Pracownik FULL JOIN Zamówienie USING (id_pracownika);
```

imię	id_zamówienia
Olga	101
Olga	102
Wiktor	103
Kasia	NULL
Lucyna	NULL
NULL	104

Zapytania zagnieźdżone
w klauzuli WHERE

Zapytania zagnieżdżone w klauzuli WHERE

Oto ogólna postać zapytania, w którym zapytanie zagnieżdżone występuje w klauzuli WHERE:

SELECT ...

FROM ...

WHERE <początek warunku> (SELECT ...)

Zapytania zagnieżdżone w klauzuli WHERE

<początek warunku> może przyjmować jedną z 5 form:

<początek warunku> ::=
 <wyrażenie> <operator>
 <wyrażenie> IN
 <wyrażenie> NOT IN
 EXISTS
 NOT EXISTS

Zapytania zagnieżdżone w klauzuli WHERE

Na tym wykładzie omówię zapytania zagnieżdżone typu

WHERE <wyrażenie> <operator> (SELECT ...)

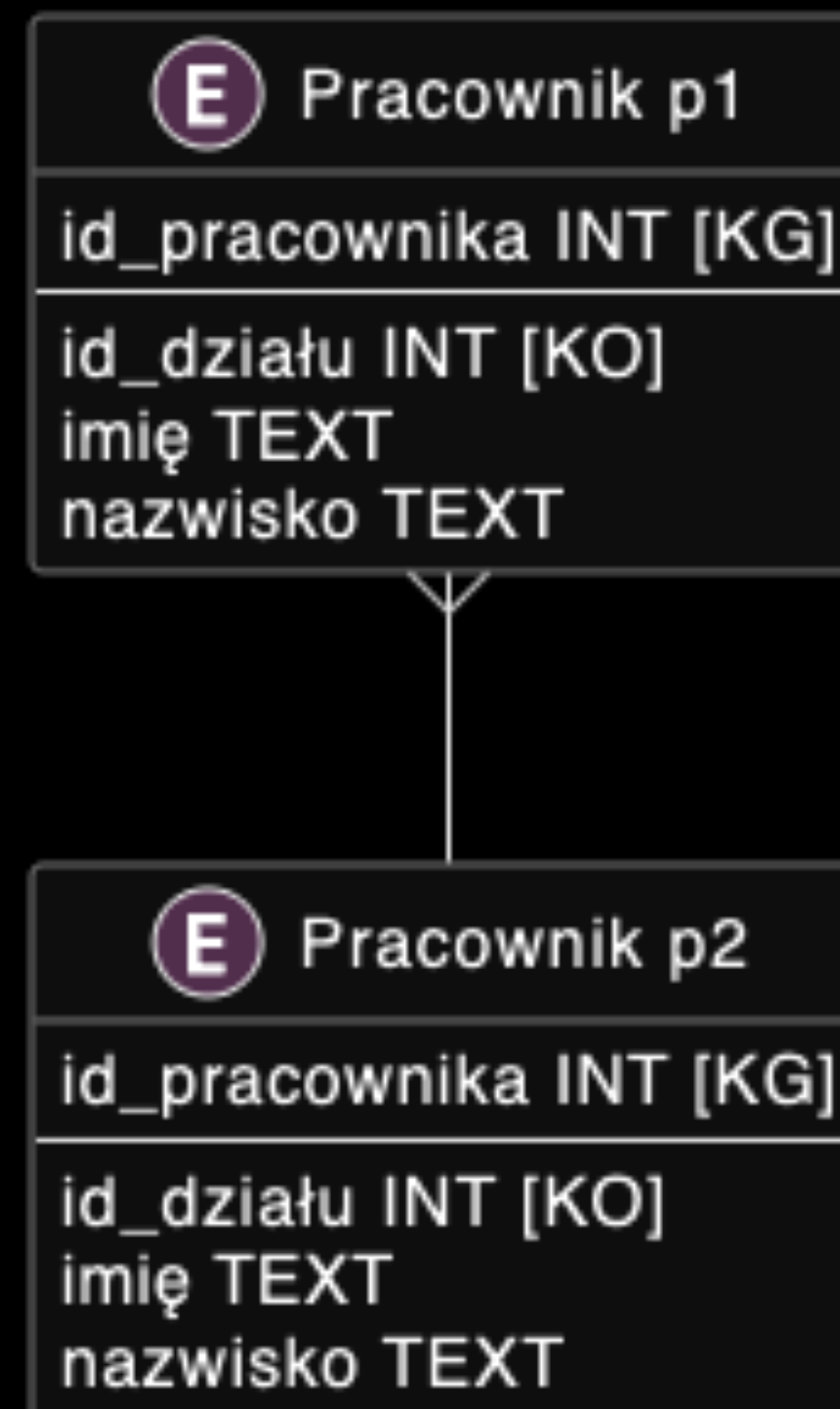
Wynikiem zapytania zagnieżdżonego powinien być co najwyżej 1 wiersz

Zapytania zagnieżdżone w klauzuli WHERE

Przykład 1: Znaleźć imiona tych pracowników, którzy pracują w tym samym dziale, co Kasia

Zapytania zagnieżdżone w klauzuli WHERE

Rozwiązanie 1a: Użyjemy złączenia względem kolumny `id_działu`



Zapytania zagnieżdżone w klauzuli WHERE

```
SELECT p2.imię  
FROM Pracownik p1 JOIN Pracownik p2 USING (id_działu)  
WHERE p1.imię = 'Kasia';
```

imię
Kasia
Lucyna

Zapytania zagnieżdżone w klauzuli WHERE

Rozwiązanie 1b: Użyjemy zapytania zagnieżdżonego

```
SELECT imię  
FROM Pracownik  
WHERE id_działu = (  
    SELECT id_działu  
    FROM Pracownik  
    WHERE imię = 'Kasia');
```

imię
Kasia
Lucyna

Zapytania zagnieżdżone w klauzuli WHERE

Przykład 2: Znaleźć imię kierownika Działu
Analiz

Zapytania zagnieżdżone w klauzuli WHERE

Rozwiązanie 2a: Użyjemy złączenia tabel
Pracownik i Dział



Złączenia zewnętrzne, czyli OUTER JOIN

```
SELECT imię  
FROM Pracownik p JOIN Dział d  
    ON p.id_pracownika = d.id_pracownika_szefa_działu  
WHERE d.nazwa = 'Dział Analiz';
```

imię
Kasia

Zapytania zagnieżdżone w klauzuli WHERE

Rozwiązanie 2b: Użyjemy zapytania zagnieżdżonego

```
SELECT imię  
FROM Pracownik  
WHERE id_pracownika = (  
    SELECT id_pracownika_szefa_działu  
    FROM Dział  
    WHERE nazwa = 'Dział Analiz');
```

imię
Kasia

Zapytania zagnieżdżone w klauzuli WHERE

Przykład 3: Znaleźć tego pracownika, który dostał najwyższą wypłatę

```
SELECT imię, kwota_wypłaty_PLN  
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)  
WHERE kwota_wypłaty_PLN = (  
    SELECT MAX(kwota_wypłaty_PLN)  
    FROM PracownikZespołu);
```

imię	kwota_wypłaty_PLN
Lucyna	20000.0

Zapytania zagnieżdżone w klauzuli WHERE

Przykład 4: Znaleźć tych pracowników, którzy dostali wypłatę poniżej średniej wypłaty

```
SELECT imię, kwota_wypłaty_PLN
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)
WHERE kwota_wypłaty_PLN < (
    SELECT AVG(kwota_wypłaty_PLN)
    FROM PracownikZespołu);
```

imię	kwota_wypłaty_PLN
Wiktor	10000.0
Olga	12000.0

Zapytania zagnieżdżone w klauzuli WHERE

Przykład 5: Znaleźć tych pracowników, którzy dostali wypłatę równą najwyższej wypłacie Wiktora lub wyższą od niej

```
SELECT imię, kwota_wypłaty_PLN
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)
WHERE kwota_wypłaty_PLN >= (
    SELECT MAX(kwota_wypłaty_PLN)
    FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)
    WHERE imię = 'Wiktor');
```

imię	kwota_wypłaty_PLN
Olga	15000.0
Wiktor	15000.0
Lucyna	20000.0

Zapytania zagnieżdżone w klauzuli HAVING

Zapytania zagnieżdżone można też umieszczać w klauzuli HAVING

Zapytania zagnieżdżone w klauzuli HAVING

Przykład 6: Znaleźć tych pracowników, których suma wypłat jest większa od sumy wypłat Lucyny

```
SELECT imię, id_pracownika, SUM(kwota_wypłaty_PLN)
FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)
GROUP BY imię, id_pracownika
HAVING SUM(kwota_wypłaty_PLN) > (
    SELECT SUM(kwota_wypłaty_PLN)
    FROM Pracownik JOIN PracownikZespołu USING (id_pracownika)
    WHERE imię = 'Lucyna');
```

imię	id_pracownika	SUM(kwota_wypłaty_PLN)
Olga	1	27000.0
Wiktor	2	25000.0

To już koniec :-)

Tematy dzisiejszego wykładu

Z tego wykładu dowiedziecie się:

- + jak sortować wyniki zapytań dzięki
klauszuli ORDER BY

- + jak ograniczać liczbę wierszy w wynikach
dzięki klauszuli LIMIT

- + jak pisać złączenia zewnętrzne, czyli
OUTER JOIN

- + jak pisać zapytania zagnieżdżone

Proszę o pytania :-)

Zapraszam was na następny wykład. Dowiedcie się z niego:

- + o klauzulach UNION i UNION ALL
- + o klauzuli INTERSECT
- + o klauzuli EXCEPT
- + o wspólnych wyrażeniach tabelowych (Common Table Expressions, CTE), czyli o klauzuli WITH
- + o innych rodzajach zapytań zagnieżdżonych

Podziękowania

Podziękowania —<—+—@

Dziękuję:

+ prof. Stanisławowi Koziełskiemu

Źródła grafik

+ <https://gemini.google.com/>